

Entrega Parcial TFG

Módulo de Autenticación

Alumno: Javier Celia

Fecha: 29 de abril de 2026

Proyecto: Djinni — Plataforma TTRPG Online

Repositorio: Djinni-FB (monorepo: `Djinni-B/` backend Symfony · `Djinni-F/` frontend React)

1. Página de Inicio (Home)

La ruta raíz `/` renderiza el componente `MainLayout`, que sirve como punto de entrada principal de la plataforma. Presenta la interfaz de bienvenida con el layout completo de la aplicación: cabecera con logo Djinni y enlaces de navegación (*Login / Register*), el formulario de acceso centrado en pantalla, y pie de página con secciones informativas. La paleta verde oscuro y el tipado Tailwind CSS son consistentes en toda la aplicación.

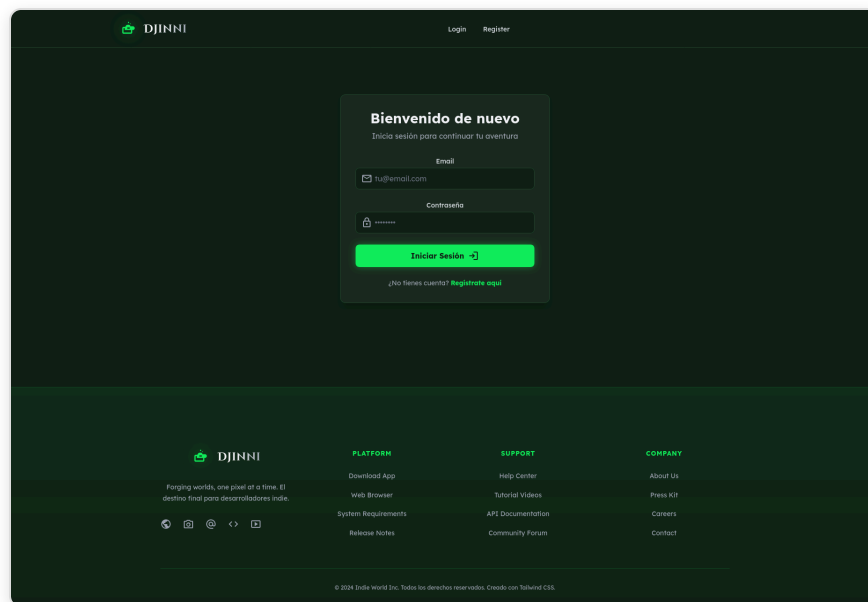
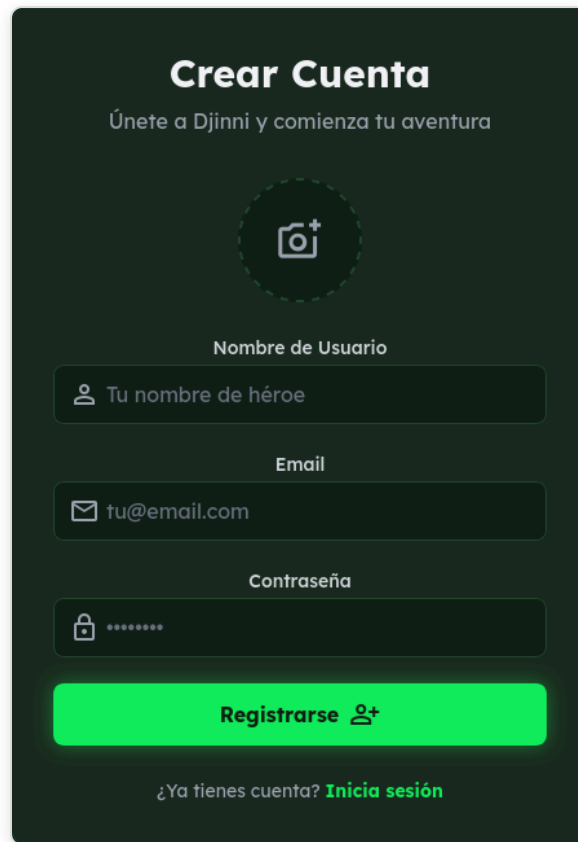


Fig. 1 — Página de inicio (ruta `/`) con layout completo: cabecera, contenido y footer.

2. Formulario de Registro

2.1 Formulario vacío

El formulario de registro (`/register`) muestra los campos requeridos: nombre de usuario, email y contraseña. Incluye un área interactiva para subir avatar (opcional). El diseño sigue la guía visual de la plataforma con fondo verde oscuro y botón de acción en verde brillante.



El formulario de registro, titulado "Crear Cuenta", tiene un fondo verde oscuro. En la parte superior, el título "Crear Cuenta" está en blanco, seguido del subtítulo "Únete a Djinni y comienza tu aventura" en un gris más claro. Debajo de esto hay un ícono de cámara dentro de un círculo verde punteado. Los campos de entrada están agrupados y tienen títulos en gris claro: "Nombre de Usuario" con el placeholder "Tu nombre de héroe", "Email" con el placeholder "tu@email.com", y "Contraseña" con caracteres ocultos por puntos. El botón "Registrarse" es de un verde brillante y lleva un ícono de usuario. En la parte inferior, hay un enlace "¿Ya tienes cuenta? Inicia sesión" en verde claro.

Fig. 2 — Formulario de registro vacío (ruta `/register`).

2.2 Formulario con datos válidos antes de enviar

Los campos corresponden exactamente a los atributos de la entidad `User` : `username` , `email` y `password` . El campo avatar, aunque opcional, también está contemplado en el modelo de datos (`avatar_url`).

Crear Cuenta

Únete a Djinni y comienza tu aventura



Nombre de Usuario

 porrete

Email

 porro@gmail.com

Contraseña

 ...

Registrarse 

¿Ya tienes cuenta? [Inicia sesión](#)

Fig. 3 — Formulario relleno con datos válidos: username porrete, email porro@gmail.com.

2.3 Registro exitoso

Tras un registro correcto, el backend responde con HTTP `201 Created` y la aplicación redirige automáticamente a `/login`. El usuario puede iniciar sesión de inmediato con las credenciales recién creadas.

Bienvenido de nuevo

Inicia sesión para continuar tu aventura

Email

Contraseña

Iniciar Sesión →

¿No tienes cuenta? [Regístrate aquí](#)

Fig. 4 — Redirección a `/login` tras registro exitoso.

2.4 Error de validación — email ya en uso (opcional)

Si el email ya existe en la base de datos, el backend responde con HTTP `409 Conflict`. El frontend captura el error y muestra el mensaje inline: *"Error al registrarse. El email podría estar en uso."*

Crear Cuenta

Únete a Djinni y comienza tu aventura

⌚

Error al registrarse. El email podría estar en uso.

Nombre de Usuario

porrete

Email

porro@gmail.com

Contraseña

...

Registrarse

¿Ya tienes cuenta?

Inicia sesión

Fig. 5 — Mensaje de error cuando el email ya está registrado.

3. Formulario de Login

3.1 Formulario vacío

El formulario de login (`/login`) solicita email y contraseña. El diseño minimalista centra la atención en las credenciales, con enlace directo a registro para nuevos usuarios.

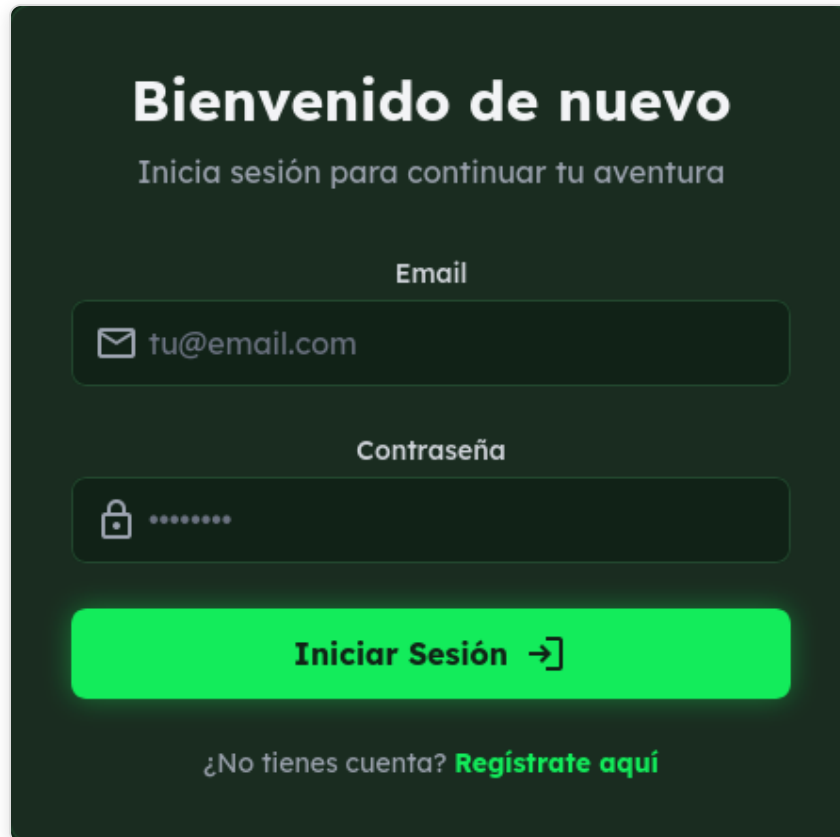
The image shows a login form on a dark green background. At the top, the text 'Bienvenido de nuevo' is displayed in a large, bold, white font, followed by 'Inicia sesión para continuar tu aventura' in a smaller, lighter font. Below this, there are two input fields. The first is labeled 'Email' and contains the text 'tu@email.com' with an envelope icon on the left. The second is labeled 'Contraseña' and contains a series of dots with a lock icon on the left. Below the input fields is a large, bright green button with the text 'Iniciar Sesión →'. At the bottom, there is a link that says '¿No tienes cuenta? Regístrate aquí'.

Fig. 6 — Formulario de login vacío (ruta `/login`).

3.2 Credenciales válidas antes de enviar

El formulario con el email del usuario introducido, listo para ser enviado al endpoint `POST /api/login_check`. La contraseña permanece enmascarada.

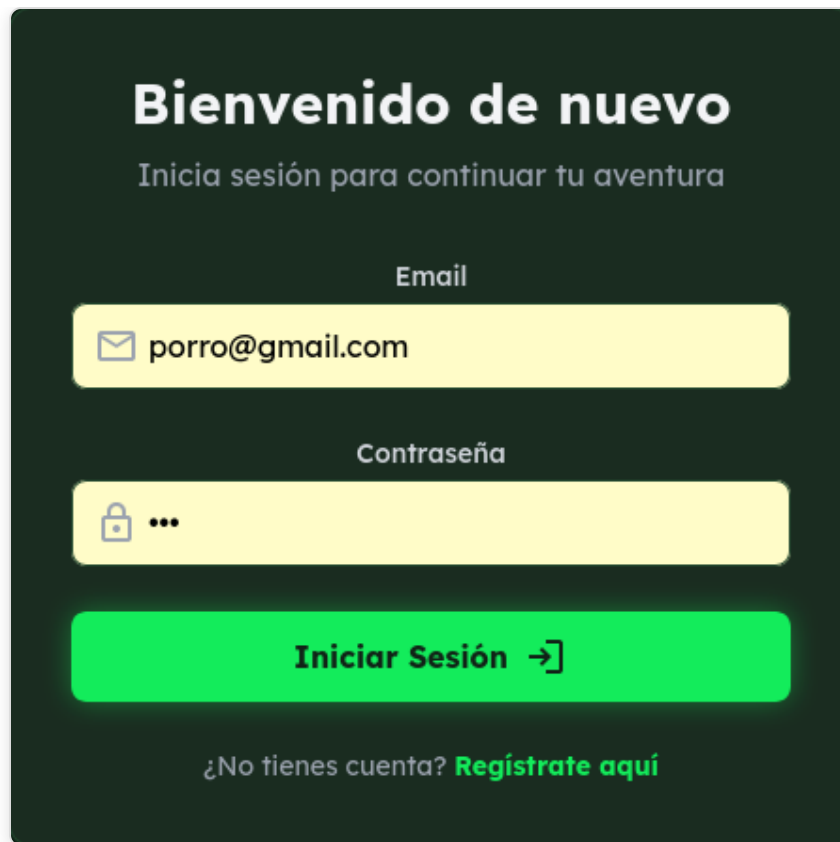


Fig. 7 — Login con credenciales válidas antes de enviar.

3.3 Panel de usuario tras login exitoso

Tras autenticarse, el servidor emite un JWT firmado. El cliente lo almacena en `localStorage` (`vtt_token`) y redirige a `/Games`. La cabecera refleja el estado autenticado: aparecen las secciones *Games*, *Character*, *Monster*, *Logout* y el botón **Create Game**, así como el avatar del usuario.

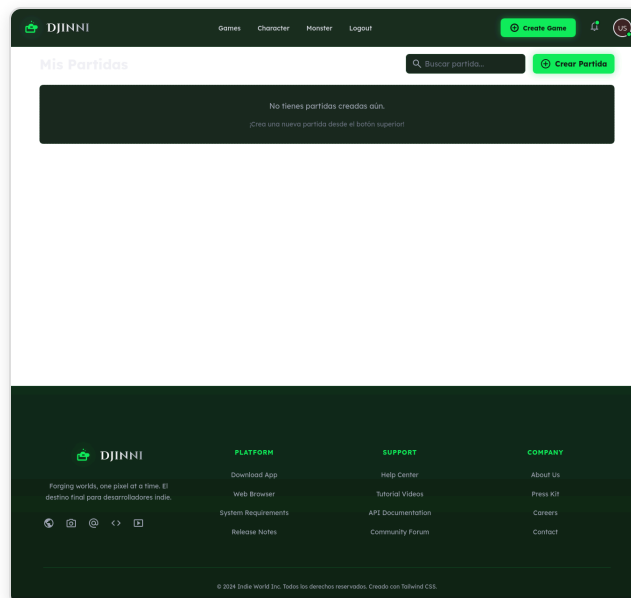
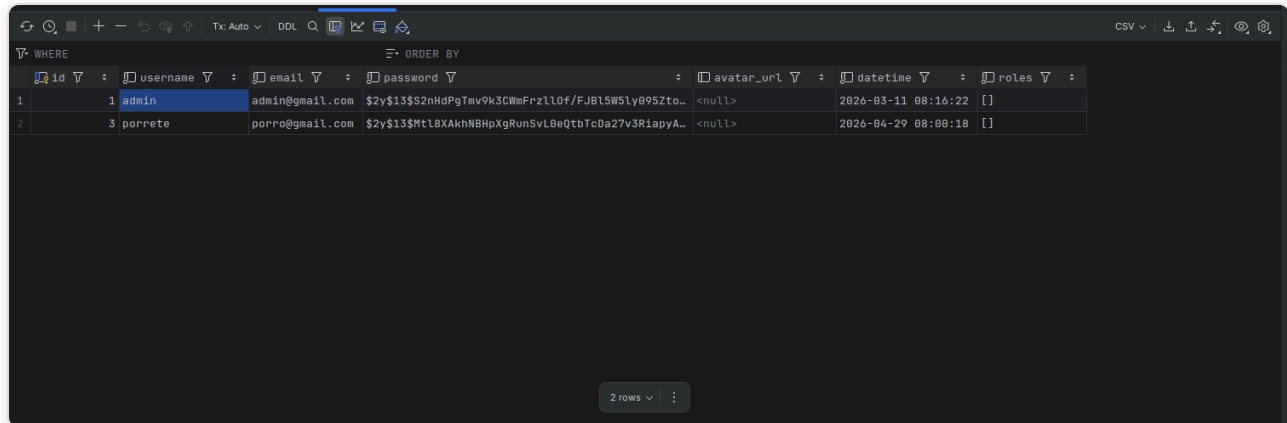


Fig. 8 — Página `/Games` tras login exitoso. Header autenticado con opciones de usuario.

4. Evidencia de Conexión con la Base de Datos

La captura muestra la tabla `user` en MariaDB visualizada con **DBeaver**. Se observan dos usuarios registrados. El usuario `porrete` (`porro@gmail.com`) fue creado el 29/04/2026 a las 08:00 a través del formulario de registro de la aplicación. Las contraseñas están almacenadas como hashes **bcrypt** (`$2y$13$...`), nunca en texto plano. Los campos coinciden exactamente con la entidad `User.php`: `id`, `username`, `email`, `password`, `avatar_url`, `datetime`, `roles`.



	id	username	email	password	avatar_url	datetime	roles
1	1	admin	admin@gmail.com	\$2y\$13\$S2nHdPgTmv9k3CwmFrzll0f/FJ8l5W5ly0952to..	<null>	2026-03-11 08:16:22	[]
2	3	porrete	porro@gmail.com	\$2y\$13\$Mt18XAKhNBHpXgRunSvL0eQtbtC0aZ7v3R1apyA..	<null>	2026-04-29 08:00:18	[]

Fig. 9 — Tabla `user` en MariaDB (DBeaver): 2 filas con contraseñas hashadas en `bcrypt`.

5. Descripción Técnica

5.1 Entidad Doctrine: `User.php`

La entidad principal del módulo es `App\Entity\User`, ubicada en `Djinni-B/src/Entity/User.php`. Implementa `UserInterface` y `PasswordAuthenticatedUserInterface` de Symfony Security.

Campo	Tipo Doctrine	Descripción
<code>id</code>	integer (PK, auto)	Clave primaria autoincremental
<code>username</code>	string(255)	Nombre visible del usuario
<code>email</code>	string(255)	Identificador único; usado en <code>getUserIdentifier()</code>
<code>password</code>	string(255)	Hash bcrypt de la contraseña
<code>avatar_url</code>	string(255), nullable	Ruta relativa a la imagen de perfil
<code>datetime</code>	datetime_immutable	Fecha y hora de registro
<code>roles</code>	json	Array de roles; <code>ROLE_USER</code> siempre incluido

Relaciones ORM configuradas:

- `OneToMany` → `UserGameSession`: sesiones de juego en las que participa
- `OneToMany` → `Monster`: monstruos creados por el usuario
- `OneToMany` → `MonsterUser`, `CharacterSheetUser`

5.2 Controladores y Rutas

Registro — `App\Controller\control_usuario\RegisterController`:

```
#[Route('/api/register', name: 'api_register', methods: ['POST'])]
public function register(
    Request $request,
    UserPasswordHasherInterface $passwordHasher,
    EntityManagerInterface $entityManager,
    SluggerInterface $slugger
): JsonResponse
```

Recibe `multipart/form-data` con `username`, `email`, `password` y `avatar` (opcional). Flujo:

1. Valida presencia de campos obligatorios (HTTP 400 si faltan)
2. Hashea la contraseña con `UserPasswordHasherInterface::hashPassword()`

3. Si hay avatar, lo mueve a `public/uploads/avatars/` con nombre único via `SluggerInterface`
4. Persiste con `EntityManager::persist() + flush()`
5. Devuelve `201 Created` en éxito, `409 Conflict` si el email ya existe

Login — gestionado por `lexik/jwt-authentication-bundle`:

```
POST /api/login_check
Content-Type: application/json
{"email": "porro@gmail.com", "password": "..."}

```

No requiere controlador personalizado. Symfony intercepta esta ruta a través del firewall definido en `config/packages/security.yaml`. Devuelve un JWT firmado con RS256 en caso de credenciales correctas.

SecurityController (`App\Controller\SecurityController`):

- `#[Route('/login')]` — renderiza `security/login.html.twig` (contextos no-SPA)
- `#[Route('/logout')]` — interceptada por el firewall para invalidar sesión

5.3 Arquitectura Frontend — React SPA

Djinni emplea una arquitectura desacoplada: el backend Symfony actúa exclusivamente como REST API, mientras que React gestiona toda la capa de presentación. No existe un `base.html.twig` como plantilla compartida para las vistas de autenticación; en su lugar:

Elemento React	Equivalente Symfony/Twig	Ruta
<code>index.html</code> (Vite)	<code>base.html.twig</code>	—
<code>App.jsx</code>	Layout raíz + router	todas
<code>Header.jsx</code> / <code>Footer.jsx</code>	Bloques Twig comunes	todas
<code>MainLayout.jsx</code>	Vista <i>home</i>	<code>/</code>
<code>RegisterPage.jsx</code>	Vista de registro	<code>/register</code>
<code>LoginPage.jsx</code>	Vista de login	<code>/login</code>

`RegisterPage.jsx` construye un `FormData` y lo envía como `multipart/form-data` a `POST /api/register`. `LoginPage.jsx` envía JSON a `POST /api/login_check`; si el servidor responde con un JWT, éste se guarda en `localStorage` como `vtt_token` y se inyecta en todas las peticiones axios siguientes mediante un interceptor global en `axiosConfig.js`.

Las rutas privadas (`/Games`, `/play/:id`, `/Character`, `/Monster`) están protegidas por el componente `PrivateRoute`, que comprueba la existencia de `vtt_token` y redirige a `/login` si no existe.

Framework CSS: **Tailwind CSS**, integrado en `vite.config.js`.

5.4 Configuración Docker y Base de Datos

Base de datos: **MariaDB 10.11.2** levantada con Docker Compose (`Djinni-B/compose.yaml`).

Configuración relevante:

```
services:
  database:
    image: mariadb:10.11.2
    environment:
      MARIADB_DATABASE: app
      MARIADB_USER: app
      MARIADB_PASSWORD: !ChangeMe!
    ports:
      - "3306:3306"
```

Cadena de conexión en `Djinni-B/.env` :

```
DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app
              ?serverVersion=10.11.2-MariaDB&charset=utf8mb4"
```

El esquema de la tabla `user` fue generado por Doctrine Migrations a partir de la entidad `User.php` . Para aplicar las migraciones:

```
php bin/console doctrine:migrations:migrate
```

El servicio `mailhog` también está incluido en el compose para capturar emails en desarrollo (puerto 8025).

El servicio `mercure` gestiona los eventos en tiempo real del tablero virtual.

6. Píldoras de Investigación

Seguridad en contraseñas: **bcrypt** y **hashing seguro**

Symfony utiliza por defecto el algoritmo **bcrypt** a través de `UserPasswordHasherInterface` . El prefijo `$2y$13$` visible en la captura de base de datos indica bcrypt con factor de coste 13, lo que implica aproximadamente 300 ms por operación de hash, haciendo los ataques de fuerza bruta computacionalmente inviables.

El *salting* es automático: cada hash incluye una sal aleatoria de 22 caracteres, por lo que dos usuarios con la misma contraseña producen hashes completamente distintos. Las contraseñas nunca se almacenan en texto plano, como demuestra la evidencia de base de datos. Actualmente, los algoritmos recomendados son **bcrypt**, **Argon2id** y **scrypt** por su resistencia a ataques con hardware especializado (GPUs/ASICs).

Manejo de sesiones y JWT en arquitecturas SPA

Djinni implementa autenticación **stateless** mediante JWT (JSON Web Tokens, RFC 7519) con el bundle `lexik/jwt-authentication-bundle`. Tras el login, el backend emite un token firmado con clave RSA privada (RS256); el frontend lo almacena en `localStorage` y lo adjunta en cada petición como `Authorization: Bearer <token>`.

JWT vs Sesiones tradicionales: las sesiones en cookie requieren estado en el servidor (tabla de sesiones o caché). Los JWT son autocontenidos y verificables sin consultar la base de datos, lo que facilita la escalabilidad horizontal y es idiomático en arquitecturas SPA + REST API como la de este proyecto.